FactSet PA Engine API: Metric Examples

In-depth request/response walkthroughs for characteristics, returns, and attribution — then a side-by-side comparison against DD Lens (Mosaic)

Part 3 of this study series. Parts 1–2 covered general calculator-API patterns and the full PACalculationParameters field reference; this document builds worked examples on top of that schema and contrasts it with Mosaic's own calculator-centric, dataBundles-based design.

A note on the examples below: FactSet does not publish literal sample result payloads for specific metrics publicly. The request bodies in this document use real, documented field names and real endpoint paths (Parts 1–2). The result bodies are **illustrative reconstructions**, built from the documented output concepts — row/column organization, statistics-per-column, a date axis — not verbatim vendor output. Every illustrative block is labeled as such.

Contents

Section	Page
1. Architecture Recap (FactSet PA Engine)	3
2. Worked Examples by Metric (Characteristics, Returns, Attribution, Risk, Batch)	3
3. Mosaic (DD Lens) — Architecture Recap & Worked Examples	8
4. Side by Side: Architecture & Documentation	10
5. Side by Side: Request & Response Shape	11
6. Side by Side: Comparative / Benchmark Logic	12
7. Side by Side: Catalog References & Job Lifecycle	13
8. Main Differences, Summarized	14
9. Main Similarities, Summarized	14
9b. Appendix: Term Crosswalk	15
10. Takeaways for Mosaic	15

Page numbers are approximate section starting points, not live-linked.

---

1. Architecture Recap

Every PA Engine calculation follows the same four-step shape regardless of which metric you're requesting:

```
1. POST /analytics/engines/pa/v3/calculations         -> 202 Accepted, Location header
2. GET  .../calculations/{id}/status                    -> poll until unit status = Completed
3. GET  .../calculations/{id}/units/{unitId}/result      -> fetch data (format per meta.format)
4. (optional) DELETE .../calculations/{id}               -> cancel if no longer needed
```

What changes between characteristics, returns, and attribution is only the **componentid** (which saved PA report template you're pointing at) and which optional override fields are relevant — columns/statistics for characteristics, dates.frequency for a returns time series, benchmarks + groups for attribution. The envelope, the polling contract, and the error shape are identical across all three. This is the same "one engine, many report types" idea noted in Part 1 — the examples below make it concrete.

Structure per Parts 1–2 of this series, grounded in github.com/FactSet/enterprise-sdk (PA Engine v3, API v3.23.0).

It's worth restating why componentid carries so much weight in every example that follows. A PA "component" is a saved report design — created and configured once, typically by a business user working in the PA desktop GUI, then given a path like pa3_documents:/pa_api_default_document-rbics. The API request doesn't describe the report from scratch; it points at that saved design and supplies only the deltas (which account, which dates, which column overrides) for this particular run. That's why the four metric examples in Section 2 look so structurally similar to each other — characteristics, returns, attribution, and risk aren't four different API shapes, they're four different *components* being run through the same override envelope.

---

2. Worked Examples by Metric

2.1 Characteristics — sector & country exposure

Request — a single-unit batch asking for security-level detail, grouped implicitly by the saved component's sector/country breakdown:

```
POST /analytics/engines/pa/v3/calculations
```

```
{
  "data": {
    "char_run": {
      "componentid": "pa3_documents:/pa_api_default_document-rbics",
      "accounts": [ { "id": "Client:/portfolios/100D-GB.acct", "holdingsmode": "B&H" } ],
      "dates": { "startdate": "20260630", "enddate": "20260630" },
      "columns": [
        { "id": "SECTOR-WEIGHT",  "statistics": ["PortfolioWeight", "BenchmarkWeight", "ActiveWeight"] },
        { "id": "COUNTRY-WEIGHT", "statistics": ["PortfolioWeight", "ActiveWeight"] }
      ],
      "currencyisocode": "USD",
      "componentdetail": "GROUPS"
    }
  },
  "meta": { "contentorganization": "SimplifiedRow", "contenttype": "Json", "format": "JsonStach" }
}
```

Status poll once the unit completes:

```
GET /analytics/engines/pa/v3/calculations/{id}/status
{ "calculationid": "{id}", "status": "Completed",
  "units": { "char_run": { "status": "Completed", "progress": "100", "points": 2,
    "result": "/analytics/engines/pa/v3/calculations/{id}/units/char_run/result",
    "warnings": [], "errors": [] } } }
```

Result (illustrative, row-organized JSON for a single snapshot date — a real JsonStach payload is denser/columnar; this shows the same information decoded to rows for readability):

```
GET .../units/char_run/result
{
  "date": "2026-06-30",
  "groups": [
    { "dimension": "SECTOR-WEIGHT", "label": "Information Technology",
      "portfolioWeight": 28.41, "benchmarkWeight": 24.90, "activeWeight": 3.51 },
    { "dimension": "SECTOR-WEIGHT", "label": "Health Care",
      "portfolioWeight": 12.03, "benchmarkWeight": 13.20, "activeWeight": -1.17 },
    { "dimension": "COUNTRY-WEIGHT", "label": "United States",
      "portfolioWeight": 71.85, "activeWeight": 6.02 }
  ]
}
```

Field names (columns/statistics/componentdetail) are documented per Part 2; the result shape is illustrative.

2.2 Returns / Performance — monthly time series

Request — a returns time series over six months, monthly frequency, portfolio vs. benchmark:

```
POST /analytics/engines/pa/v3/calculations
{
  "data": {
    "returns_run": {
      "componentid": "pa3_documents:/pa_api_default_document-performance",
      "accounts":   [ { "id": "Client:/portfolios/100D-GB.acct", "holdingsmode": "TBR" } ],
      "benchmarks": [ { "id": "BENCH:RUSSELL-1000" } ],
      "dates": { "startdate": "20260101", "enddate": "20260630", "frequency": "Monthly", "calendar": "FIVEDAY" },
      "columns": [ { "id": "TOTAL-RETURN", "statistics": ["PortfolioReturn", "BenchmarkReturn", "ActiveReturn"] } ],
      "currencyisocode": "USD",
      "componentdetail": "TOTALS"
    }
  },
  "meta": { "contentorganization": "SimplifiedRow", "contenttype": "Json", "format": "JsonStach" }
}
```

Result (illustrative) — note the shape change from Section 2.1: characteristics returned one row per group at a single date; a time series returns one row per date with the same columns repeated:

```
{
  "series": [
    { "date": "2026-01-31", "portfolioReturn": 2.14, "benchmarkReturn": 1.87, "activeReturn": 0.27 },
    { "date": "2026-02-28", "portfolioReturn": -0.62, "benchmarkReturn": -0.41, "activeReturn": -0.21 },
    { "date": "2026-03-31", "portfolioReturn": 3.05, "benchmarkReturn": 2.66, "activeReturn": 0.39 },
```

```
    { "date": "2026-04-30", "portfolioReturn": 1.18, "benchmarkReturn": 1.02, "activeReturn": 0.16 },
    { "date": "2026-05-31", "portfolioReturn": -1.44, "benchmarkReturn": -1.10, "activeReturn": -0.34 },
    { "date": "2026-06-30", "portfolioReturn": 2.77, "benchmarkReturn": 2.30, "activeReturn": 0.47 }
  ],
  "cumulative": { "portfolioReturn": 7.31, "benchmarkReturn": 6.61, "activeReturn": 0.70 }
}
```


This is the clearest illustration of why dates.frequency is on the shared PADateParameters object rather than a metric-specific field (Part 2, Section 4.2): the exact same field flips a characteristics snapshot into a returns time series just by being present.

2.3 Attribution — Brinson-style sector allocation/selection

Request — groups by sector, columns request allocation and selection effects, one unit in the same batch pattern as Part 2 Section 6:

```
POST /analytics/engines/pa/v3/calculations
{
  "data": {
    "attr_run": {
      "componentid": "pa3_documents:/pa_api_default_document-attribution",
      "accounts":   [ { "id": "Client:/portfolios/100D-GB.acct", "holdingsmode": "TBR" } ],
      "benchmarks": [ { "id": "BENCH:RUSSELL-1000" } ],
      "dates": { "startdate": "20260101", "enddate": "20260630", "frequency": "Monthly" },
      "groups": [ { "id": "GICS-SECTOR", "frequency": "Monthly" } ],
      "columns": [
        { "id": "ATTR-EFFECTS", "statistics": ["AllocationEffect", "SelectionEffect", "InteractionEffect", "TotalEffect"] }
      ],
      "datasources": { "useportfoliopricingsourcesforbenchmark": true },
      "currencyisocode": "USD",
      "componentdetail": "GROUPS"
    }
  },
  "meta": { "contentorganization": "SimplifiedRow", "contenttype": "Json", "format": "JsonStach" }
}
```

Result (illustrative) — attribution combines the grouped shape from 2.1 with the date-series shape from 2.2, plus an extra effect breakdown per group per date:

```
{
  "period": { "startdate": "2026-01-01", "enddate": "2026-06-30" },
  "groups": [
    { "sector": "Information Technology",
      "allocationEffect": 0.18, "selectionEffect": 0.41, "interactionEffect": 0.03, "totalEffect": 0.62 },
    { "sector": "Health Care",
      "allocationEffect": -0.07, "selectionEffect": 0.12, "interactionEffect": -0.01, "totalEffect": 0.04 },
    { "sector": "Financials",
      "allocationEffect": 0.02, "selectionEffect": -0.09, "interactionEffect": 0.00, "totalEffect": -0.07 }
  ],
  "totalActiveReturn": 0.70
}
```

Note totalActiveReturn in the attribution result (0.70) matches the cumulative active return from the returns example in 2.2 — attribution is, by construction, a decomposition of the same active return into sector-level effects. That reconciliation is exactly why FactSet keeps benchmarks, dates, and datasources as shared, reusable sub-objects (Part 2, Section 4) rather than attribution-specific fields: the same benchmark/date/pricing context has to hold across both calls for the numbers to tie out.

2.4 Predicted Risk — tracking error & factor exposure

Request — same envelope again, this time with a component built for predicted (ex-ante) risk rather than realized performance. Note there is no full date range: a predicted-risk snapshot is as-of a single date, another example of the shared PADateParameters object flexing to fit the metric rather than needing a risk-specific date object:

```
POST /analytics/engines/pa/v3/calculations
{
  "data": {
    "risk_run": {
      "componentid": "pa3_documents:/pa_api_default_document-predicted-risk",
      "accounts":   [ { "id": "Client:/portfolios/100D-GB.acct", "holdingsmode": "B&H" } ],
      "benchmarks": [ { "id": "BENCH:RUSSELL-1000" } ],
      "dates": { "enddate": "20260630" },
      "columns": [
        { "id": "RISK-SUMMARY", "statistics": ["TrackingError", "PortfolioVolatility", "BenchmarkVolatility", "Beta"] },
        { "id": "FACTOR-EXPOSURE", "statistics": ["ActiveExposure"] }
      ],
      "currencyisocode": "USD",
      "componentdetail": "TOTALS"
    }
  },
  "meta": { "contentorganization": "SimplifiedRow", "contenttype": "Json", "format": "JsonStach" }
}
```

Result (illustrative):

```
{
  "date": "2026-06-30",
  "summary": { "trackingError": 2.14, "portfolioVolatility": 13.02, "benchmarkVolatility": 12.40, "beta": 1.04 },
  "factorExposures": [
    { "factor": "Value",     "activeExposure": -0.18 },
    { "factor": "Momentum",  "activeExposure":  0.22 },
    { "factor": "Size",      "activeExposure": -0.05 }
  ]
}
```

2.5 One request, four metrics — the batch pattern in practice

Sections 2.1–2.4 can all be submitted as a single POST, since data is a dictionary of independently-keyed units (Part 2, Section 2). This is the realistic shape of a client dashboard request that needs characteristics, returns, attribution, and risk for the same portfolio and date range in one round trip:

```
POST /analytics/engines/pa/v3/calculations
{
  "data": {
    "char_run": { "componentid": "...-rbics", ... },
    "returns_run": { "componentid": "...-performance", ... },
    "attr_run": { "componentid": "...-attribution", ... },
    "risk_run": { "componentid": "...-predicted-risk", ... }
  },
  "meta": { "contentorganization": "SimplifiedRow", "contenttype": "Json", "format": "JsonStach" }
}
-> 202 Accepted, Location: /analytics/engines/pa/v3/calculations/{id}

GET .../calculations/{id}/status
{ "calculationid": "{id}", "status": "Executing",
  "units": {
    "char_run":    { "status": "Completed", "result": ".../units/char_run/result" },
    "returns_run": { "status": "Completed", "result": ".../units/returns_run/result" },
    "attr_run":    { "status": "Executing", "progress": "60" },
    "risk_run":    { "status": "Queued" }
  } }
```

Each unit resolves independently and on its own timeline — a dashboard can render characteristics and returns the moment they're done while attribution and risk are still computing, rather than blocking the whole UI on the slowest metric in the batch.

---

3. Mosaic (DD Lens) — Architecture Recap

Mosaic is the Core Platform team's own API gateway: a Python/FastAPI service that consolidates roughly 20+ legacy endpoints into one unified data gateway, acting as a "storefront" orchestration layer on top of three internal services — Core Meta, Core Characteristics, and Core Returns.

- **Calculator-centric, dataBundles architecture** — chosen deliberately over a field-centric/GraphQL alternative, because allocation data's hierarchical structure was a dealbreaker for a flat field-centric model.
- **Fixed JSON hierarchy:** data → userKey → CalculatorSlug, treated as immutable — "never flatten it."
- **Three standardized response shapes:** Allocation, Time Series, Entity — every calculator's output is one of these three, not an open-ended per-metric schema.
- **Merge strategies:** join-by-shape and side-by-side, with user-defined merge keys giving explicit control over which calculators combine into one bundle.
- **Comparative logic:** mainIds establish the alignment window; compareIds are trimmed to match it — the core mechanism for portfolio-vs-benchmark-style comparisons.
- **Naming conventions:** slug-based calculator names (e.g. performance-summary-time-series), refId for bundle reference, investmentEntityId for entity identification.

Grounded in Mosaic project notes (architecture decisions, key learnings, ways-of-working).

3.1 Window vs. aggregations — a distinction Mosaic keeps explicit

Two concepts that are easy to conflate are kept deliberately separate: window describes the scope of data *availability* (what date range exists to draw from), while aggregations describes calculations that carry their own parameters and date ranges on top of that window. Aggregation functions themselves split two ways: point-in-time (no window required) vs. time-aggregated (require an explicit window), and domain-specific vs. non-domain-specific. This mirrors, in spirit, the way FactSet separates PADateParameters (the calculation's own date scope) from componentdetail (how much the result is rolled up) — both systems draw a line between "how much data underlies this" and "how is it being computed/rolled up," they just name the line differently.

3.2 Worked example — Allocation-shape request (characteristics equivalent)

Illustrative, built from Mosaic's documented naming conventions (slug-based calculator names, refId, investmentEntityId) and its data → userKey → CalculatorSlug hierarchy — the closest structural analogue to the FactSet characteristics call in Section 2.1:

```
POST /mosaic/v1/bundles
{
  "data": {
    "dashboardWidget1": {
      "sector-country-exposure": {
        "refId": "100D-GB",
        "investmentEntityId": "100D-GB",
        "window": { "asOf": "2026-06-30" },
        "aggregations": [
          { "dimension": "sector",  "measures": ["portfolioWeight", "benchmarkWeight", "activeWeight"] },
          { "dimension": "country", "measures": ["portfolioWeight", "activeWeight"] }
        ]
      }
    }
  }
}
```

Result (illustrative) — the Allocation shape:

```
{
  "data": {
    "dashboardWidget1": {
      "sector-country-exposure": {
        "shape": "Allocation",
        "asOf": "2026-06-30",
```

```
        "groups": [
          { "dimension": "sector", "label": "Information Technology",
            "portfolioWeight": 28.41, "benchmarkWeight": 24.90, "activeWeight": 3.51 },
          { "dimension": "country", "label": "United States",
            "portfolioWeight": 71.85, "activeWeight": 6.02 }
        ]
      }
    }
  }
}
```

Set beside Section 2.1's FactSet result, the information content is nearly identical — a list of dimension/label/weight rows — but Mosaic's response wraps it in an explicit "shape": "Allocation" tag and preserves the full userKey/CalculatorSlug path from the request in the response. FactSet's result, by contrast, is addressed by URL (the per-unit result endpoint) rather than self-labeled inside the payload.

3.3 Worked example — Time Series shape with comparative alignment

This mirrors the FactSet returns example (2.2), using Mosaic's mainIds/compareIds comparative mechanism in place of FactSet's parallel accounts/benchmarks arrays:

```
POST /mosaic/v1/bundles
{
  "data": {
    "dashboardWidget2": {
      "performance-summary-time-series": {
        "mainIds": ["100D-GB"],
        "compareIds": ["RUSSELL-1000"],
        "window": { "start": "2026-01-01", "end": "2026-06-30" },
        "aggregations": [ { "frequency": "monthly", "measures": ["totalReturn"] } ]
      }
    }
  }
}
```

Result (illustrative) — the Time Series shape, with compareIds trimmed to the mainIds window per the documented alignment rule:

```
{
  "data": {
    "dashboardWidget2": {
      "performance-summary-time-series": {
        "shape": "Time Series",
        "series": [
          { "date": "2026-01-31", "id": "100D-GB",      "totalReturn": 2.14 },
          { "date": "2026-01-31", "id": "RUSSELL-1000",  "totalReturn": 1.87 },
          { "date": "2026-02-28", "id": "100D-GB",      "totalReturn": -0.62 },
          { "date": "2026-02-28", "id": "RUSSELL-1000",  "totalReturn": -0.41 }
        ]
      }
    }
  }
}
```

Where FactSet's returns result (2.2) pre-computes activeReturn as its own column, this illustrative Mosaic shape keeps portfolio and benchmark as separate rows tagged by id — consistent with mainIds/compareIds being distinct, explicitly-labeled roles (Section 6) rather than two columns of the same row. Deriving an active-return column from this would be exactly the kind of thing a join-by-shape merge (Section 3.4) exists to do generically.

3.4 Worked example — merging two bundles (no FactSet equivalent)

This is the one capability in these worked examples with no structural counterpart on the FactSet side (Section 5.2 discusses why): requesting two calculators and merging them by a shared key, illustrative per the documented join-by-shape / user-defined merge key mechanism:

```
POST /mosaic/v1/bundles
{
```

```
  "data": {
    "widget3": {
      "sector-exposure":      { "refId": "100D-GB", "window": { "asOf": "2026-06-30" } },
      "sector-attribution":   { "refId": "100D-GB", "compareIds": ["RUSSELL-1000"],
                                "window": { "start": "2026-01-01", "end": "2026-06-30" } }
    }
  },
  "merge": {
    "strategy": "join-by-shape",
    "mergeKeys": ["sector"],
    "groups": [ ["sector-exposure", "sector-attribution"] ]
  }
}
```

No FactSet request in Sections 2.1–2.5 has an analogue to the top-level merge block: each PA Engine unit is fetched and interpreted independently by the caller, whereas Mosaic can be asked to perform the join server-side, provided both calculators' outputs share the Allocation shape and a common sector key.

---

4. Side by Side: Architecture

Dimension	FactSet PA Engine API	Mosaic (DD Lens)
What it fronts	PA/SPAR/Vault calculation engines directly	Three internal services: Core Meta, Core Characteristics, Core Returns
Organizing principle	One engine, many saved report templates (components)	One gateway, many named calculators (slugs)
Why this shape	Components let a business user's saved report design drive the calculation without the API needing per-metric endpoints	Field-centric/GraphQL was evaluated and rejected: allocation data's hierarchy doesn't fit a flat model
Tech stack	Multi-language auto-generated SDKs (Python, .NET, Java, R) over a REST/OpenAPI service	Python, FastAPI service layer, Polars for data processing
Consumers	External enterprise clients via SDK or raw REST	Internal frontend developers and business users at DFA
Scale of consolidation	N/A — is itself the calculation engine	Consolidates ~20+ legacy endpoints into one gateway

The "why this shape" row is the one worth dwelling on. FactSet's component model exists because PA Engine serves business users who design reports visually first (in the PA GUI) and only later consume them programmatically — the API has to honor a report design that predates the API call. Mosaic has no equivalent legacy GUI to stay compatible with; its calculator-centric model was chosen directly against a field-centric/GraphQL alternative specifically because DFA's allocation data has a hierarchical shape that a flat field-centric API couldn't represent cleanly (Section 3) — a green-field decision rather than one inherited from an existing UI.

4.1 Documentation and developer experience

	FactSet PA Engine	Mosaic
Docs format	Auto-generated per-language SDK reference (OpenAPI-generated .md per model/endpoint)	Hand-authored Sphinx + MyST Markdown, deployed via Jenkins CI/CD to GitHub Pages
Audience split	Single audience: external developers integrating via SDK or REST	Explicitly split tracks: User Docs (external-facing, scoped to deployed milestones) vs. Internal Design (team-only)
Notable tooling	Multi-language SDK generation (Python, .NET, Java, R) from one OpenAPI spec	Hover previews (Tippy.js), image zoom, tab-sets, Mermaid diagrams, custom Sphinx extensions, a team logo

Onboarding path	Not documented in what's covered here	Dedicated Getting Started track: Start Here, 5 Min Intro, 15 Min Deep Dive

This is a genuine point of pride worth naming plainly: Mosaic's documentation investment — the persona-split structure, the onboarding ladder, the custom Sphinx tooling — is more deliberately designed for a mixed audience of business users and frontend developers than what's visible in FactSet's auto-generated, developer-only reference docs. FactSet's docs are thorough and consistent (a direct benefit of being machine-generated from one OpenAPI spec across four languages) but they document the API, not the domain; Mosaic's docs are explicitly trying to do both at once.

---

5. Side by Side: Request & Response Shape

5.1 Top-level JSON hierarchy

	FactSet PA Engine	Mosaic
Request keying	data → {callerDefinedUnitKey} → PACalculationParameters	data → {userKey} → {CalculatorSlug}
Is it fixed?	Flexible — caller invents the unit key per batch	Fixed and treated as immutable — "never flatten it"
Metric selection	componentid (a saved template reference) + column/group overrides	CalculatorSlug itself (e.g. performance-summary-time-series) names the metric

This is the single most structurally significant difference between the two systems. FactSet's componentid points at a server-side saved report design and the request body layers *overrides* on top of it (Part 2, Section 3) — the metric being computed is implicit in which component you reference. Mosaic's CalculatorSlug makes the metric explicit and literal in the URL-adjacent request key itself — there's no separate "saved template" indirection layer between the request and what gets computed. Put differently: FactSet's unit of reuse is a *configured report*; Mosaic's unit of reuse is a *named calculator*.

5.2 Response shape taxonomy

	FactSet PA Engine	Mosaic
Shape count	Open-ended — shape follows componentdetail (GROUPS/GROUPSALL/TOTALS/SECURITIES) and column/group config	Exactly three standardized shapes: Allocation, Time Series, Entity
Encoding density	Two formats: JsonStach (compact columnar, protobuf-derived) vs. non-Stach JSON	Not documented in what's covered here
Combining results	Not a first-class concept — caller fetches each unit's result separately	First-class: join-by-shape and side-by-side merge strategies with user-defined merge keys

The three-shapes-only constraint in Mosaic is arguably a tighter contract than FactSet's own: FactSet's shape varies with the parameters you send (Section 2 above shows three visibly different result shapes for three different metrics), whereas Mosaic deliberately collapses every calculator's output into one of Allocation, Time Series, or Entity, specifically so results are *predictable enough to merge*. That's the direct payoff of the fixed taxonomy: FactSet has no merge concept in the API itself because nothing guarantees two results share a compatible shape; Mosaic's shape discipline is what makes join-by-shape and side-by-side merging possible as generic operations rather than per-metric special cases.

6. Side by Side: Comparative / Benchmark Logic

	FactSet PA Engine	Mosaic
Mechanism	benchmarks: List<PAIdentifier> alongside accounts, resolved together by the engine	mainIds establish the alignment window; compareIds are trimmed to match it
Symmetry	Portfolio and benchmark are the same PAIdentifier type in different arrays (Part 2, Section 4.1)	Asymmetric by design: mainIds and compareIds play distinct, non-interchangeable roles
Timeline handling	Implicit — shared dates object applies to both portfolio and benchmark calculation	Explicit and named as a core differentiator: timeline alignment is handled deliberately, not implicitly
Roadmap	N/A (mature, versioned v3 API)	Explicit alignmentStrategy and includeMetrics parameters are a planned future addition

Both systems solve the same underlying problem — comparing a primary series against one or more reference series over a shared timeline — but FactSet leans on structural symmetry (same object type, shared date parameters) while Mosaic leans on explicit, asymmetric roles (mainIds vs compareIds) with alignment treated as its own named mechanism. Mosaic's approach is more self-documenting about *which side drives the window*; FactSet's is more economical in the schema but leaves the "who trims to match whom" behavior to be understood rather than named.

One practical consequence: because FactSet's benchmark is just another PAIdentifier, a calculation can technically carry multiple benchmarks in the same array with no special handling required — the engine simply computes against each. Mosaic's compareIds being a list plays the same multi-benchmark role, but because the role is named and asymmetric, it's also the natural place to later attach per-comparison behavior (like the planned alignmentStrategy) without disturbing mainIds at all — a flexibility that's harder to add cleanly to FactSet's symmetric accounts/benchmarks design without introducing a new field.

---

7. Side by Side: Catalog References & Job Lifecycle

	FactSet PA Engine	Mosaic
Metric/column catalog	Columns, groups, pricing sources, calendars are id references resolved via separate GET .../columns, .../groups, etc. lookup endpoints	CalculatorSlug is itself the catalog entry — naming convention (e.g. performance-summary-time-series) is self-describing rather than an opaque id
Async job model	Explicit: submit → 202 + Location → poll status → fetch result → optional cancel	Not documented in what's covered here — Mosaic's role as an orchestration/storefront layer over already-computed Core Platform data suggests calls may resolve more directly, but this isn't confirmed
Error model	Mature: CalculationUnitStatus.errors[] with Error.source.pointer pointing back into the request body (Part 2, Section 5.2)	Actively being designed: shape-item level errors, bundle-level error aggregation, warning deduplication, and partial upstream failure handling are open team questions
Rate/cost limiting	Two orthogonal throttles: request-rate headers plus a points budget (25/calculation, 500 across all batches)	Not documented in what's covered here

The error-model row is worth sitting with: FactSet's pointer-based error attribution (Part 2, Section 5.2) is exactly the kind of mature, field-level error reporting that Mosaic's team has flagged as still open — shape-item level errors and bundle-level aggregation are listed as unresolved questions. This is a concrete, low-risk borrow: FactSet's {id, code, title, detail, source: {pointer, parameter}} error object, adapted to point into a data.userKey.CalculatorSlug path instead of data.unitId, would directly answer the "shape-item level errors" question already on Mosaic's own list.

8. Main Differences, Summarized

- **Metric selection is implicit vs. explicit.** FactSet: componentid references a saved template; the metric is implicit in what that template was built to compute. Mosaic: CalculatorSlug names the metric directly in the request.
- **Open-ended vs. constrained response shapes.** FactSet's result shape varies by componentdetail and column/group configuration (effectively unbounded). Mosaic constrains every result to one of three shapes (Allocation, Time Series, Entity) specifically to enable generic merging.
- **No merge concept vs. first-class merge.** FactSet has no built-in way to combine two units' results; each unit is fetched and interpreted independently. Mosaic treats merging (join-by-shape, side-by-side, user-defined keys) as core functionality.
- **External enterprise API vs. internal gateway.** FactSet PA Engine is a multi-tenant, externally consumed product with mature multi-language SDKs. Mosaic is purpose-built for DFA's own frontend developers and business users, consolidating internal services rather than fronting a general-purpose calc engine.
- **Symmetric vs. asymmetric comparison roles.** FactSet treats portfolio and benchmark identically as PAIdentifier entries in parallel arrays. Mosaic names the roles explicitly — mainIds vs. compareIds — with alignment behavior as a distinct, documented mechanism.
- **Mature vs. evolving error contract.** FactSet's pointer-based per-field error model is finished and versioned (v3.23.0). Mosaic's equivalent (shape-item errors, bundle-level aggregation, warning dedup) is still an open design question.

None of these differences make one design "better" in the abstract — they're consequences of different constraints (Section 4.1). FactSet optimizes for external, multi-tenant reuse of GUI-authored report designs across four SDK languages; Mosaic optimizes for internal composability across a small, known set of Python/FastAPI consumers who need merge-able, shape-consistent results. Judged against their own goals, both hold up well.

---

9. Main Similarities, Summarized

- **Both are orchestration/gateway layers, not the data source itself.** FactSet PA Engine sits in front of PA/SPAR/Vault calculation engines; Mosaic sits in front of Core Meta/Characteristics/Returns. Neither system stores the underlying financial data — both compose and reshape it.
- **Both use a nested, keyed JSON envelope rather than flat fields.** FactSet's data → unitKey → parameters and Mosaic's data → userKey → CalculatorSlug are structurally the same idea: a dictionary keyed by a caller/request identifier, wrapping a per-item payload, deliberately chosen over a flat/field-centric alternative in both cases.
- **Both separate "what to compute" from "how it's grouped/detailed."** FactSet's groups/columns override the saved component's grouping; Mosaic's window vs. aggregations distinction (Section 3) draws the same line between data scope and computed groupings, and both are explicit about keeping the two separate rather than conflating them.
- **Both reference reusable, named building blocks rather than inlining definitions.** FactSet's column/group ids resolve against catalog lookup endpoints; Mosaic's calculator slugs and standardized shapes serve the same purpose — small stable identifiers standing in for larger, centrally defined behavior.
- **Both explicitly handle a portfolio-vs-reference comparison as a named, first-class concept** rather than leaving the caller to diff two independent results themselves (Section 6).

9b. Appendix: Term Crosswalk

A quick reference for translating between the two vocabularies used throughout this document:

Concept	FactSet term	Mosaic term
Top-level request keying	data → {unitKey}	data → {userKey}
Metric/calculation selector	componentid (saved template reference)	CalculatorSlug (e.g. performance-summary-time-series)
Portfolio reference	PAIdentifier in accounts[]	investmentEntityId / refId, or an entry in mainIds
Reference/benchmark series	PAIdentifier in benchmarks[]	entry in compareIds
Date scope	PADateParameters (startdate/enddate/frequency/calendar)	window
Computed rollups/statistics	columns[] with statistics[]; groups[]	aggregations[] with measures[]
Result shape label	Implicit in componentdetail + column/group config	Explicit: Allocation, Time Series, or Entity
Combining two results	Not a built-in concept	merge (join-by-shape / side-by-side) with mergeKeys
Preferred metric vocabulary	"statistics"	"measures" (chosen over "aggregates," aligned with Kimball/Tableau/Looker/Power BI/Fabric usage)

FactSet terms per Parts 1–2 of this series; Mosaic terms per this project's architecture notes.

10. Takeaways for Mosaic

- **Borrow the pointer-based error object wholesale.** It directly answers an open Mosaic design question (Section 7) and requires no new concepts — only mapping source.pointer onto Mosaic's own data → userKey → CalculatorSlug path instead of FactSet's data → unitId path.
- **The three-shape constraint is a genuine strength worth defending.** FactSet's own unconstrained result shapes are exactly why it has no merge primitive; Mosaic's shape discipline is the reason merging can exist as a generic feature at all. Any pressure to add a fourth ad-hoc shape for a new calculator should be weighed against this trade-off explicitly.
- **Consider naming the comparative-alignment mechanism as clearly as mainIds/compareIds already does** when the planned alignmentStrategy and includeMetrics parameters land (Section 6) — this is already Mosaic's strongest point of clarity relative to FactSet's more implicit approach, worth extending rather than diluting.
- **A points-style cost budget (Section 7) is worth evaluating** if any Mosaic-fronted calculators vary widely in backend cost the way FactSet's characteristics vs. attribution calls likely do — a flat request-rate limit under-throttles a few expensive calls and over-throttles many cheap ones.
- **The documentation persona-split (Section 4.1) is worth protecting as Mosaic scales.** FactSet's single-audience, auto-generated docs are a reasonable trade-off for an external multi-tenant product; Mosaic serving both business users and frontend developers well, in one docs site, is harder to sustain and easy to erode under deadline pressure as more calculators are added. Treat it as a maintained investment, not a one-time setup.
- **Multi-language SDK generation is likely not worth copying wholesale.** It solves a problem — serving many external client languages from one spec — that Mosaic, as an internal gateway with a known Python/FastAPI consumer base, does not currently have. Worth revisiting only if Mosaic's consumer base expands beyond DFA's own frontend stack.

Taken together, the comparison suggests Mosaic and PA Engine solved a similar problem — make a complex, multi-metric financial calculation engine consumable as clean JSON — from two different starting constraints: FactSet building outward from an existing GUI-driven report designer for external clients, Mosaic building inward to unify internal services for its own frontend teams. The convergences (Section 9) are the parts of the problem that are hard regardless of starting point — keyed envelopes, named comparison roles, catalog references over inline definitions. The divergences (Section 8) mostly trace back to that one starting-constraint difference, which is useful context when deciding whether a FactSet pattern is worth adopting on its own merits or is really just a consequence of FactSet's GUI-first history.

FactSet detail throughout is grounded in the public, open-source FactSet Enterprise SDK repository (github.com/FactSet/enterprise-sdk, PA Engine v3, API v3.23.0); illustrative result payloads in Section 2 are reconstructions for teaching purposes, not literal vendor output. Mosaic detail is drawn from this project's own architecture notes as of September 2026 and reflects points explicitly marked as still in motion where noted.